A thesis submitted in partial satisfaction of the requirements
for the degree of Master of Computer Science and Engineering
in the Graduate School of the University of Aizu

A Resilient On-Chip STDP Learning Under Memory Voltage Scaling

by

Subbaiah Ravi Hariprakash

July 2026

The thesis titled

A Resilient On-Chip STDP Learning Under Memory Voltage Scaling

by

Subbaiah Ravi Hariprakash

is reviewed and approved by:

Chief referee	
Associate Professor	Date
Khanh N. Dang	
Senior Associate Professor	Date
Okuyama Yuichi	
Assistant Professor	Date
Zhishang Wang	

THE UNIVERSITY OF AIZU

July 2026

Contents

List of Figures

List of Tables

Acknowledgment

I would like to thank my supervisor, Associate Professor Khanh N. Dang, for his guidance throughout this work. I am also grateful to Mr. Yuga Hanyu and Mr. Satvik Ganesh for their support.

Abstract

As Artificial Intelligence (AI) continues to expand, reducing its energy consumption has become increasingly important. Edge AI with on-chip learning based on Spiking Neural Networks (SNNs) offers a promising solution by enabling energy-efficient, adaptive intelligence while minimizing data movement. To further reduce power consumption, aggressively undervolted on-chip memory can be employed, but operating near-threshold voltages introduces permanent hardware faults, primarily Stuck-at-0 (SA0) and Stuck-at-1 (SA1). While SA0 faults are largely absorbed by the network, SA1 faults force neurons to fire continuously, triggering runaway lateral inhibition that permanently suppresses healthy neurons and causes catastrophic collapse of Spike-Timing-Dependent Plasticity (STDP) learning. To address this challenge, we propose STDP-DIP (Spike-Timing-Dependent Plasticity with Diagnostic Isolation Pruning), a fault-aware on-chip learning framework that integrates fault injection with STDP learning. STDP-DIP identifies SA1 faults during learning and structurally severs their inhibitory outputs, preventing pathological inhibition from propagating through the network while preserving normal neuronal dynamics and synaptic plasticity. Experimental results demonstrate that STDP-DIP restores stable STDP learning under severe near-threshold memory fault conditions, achieving $>91\%$ accuracy at 20% bit error rates, improving performance by over 80% compared with conventional uncorrected approaches while maintaining the energy benefits of near-threshold operation.

Chapter 1

Introduction

1.1 Background: The AI Energy Crisis

The rapid adoption of Artificial Intelligence (AI) has made energy efficiency a critical challenge [1, 2]. A significant portion of AI energy consumption originates from the memory subsystem, where frequent data movement between processors and memory dominates both latency and power [3]. This growing performance and energy gap, commonly referred to as the "Memory Wall," has become a major obstacle to scaling conventional AI systems based on high-precision DRAM [4]. The challenge is even more pronounced in edge and IoT devices, where memory capacity and energy budgets are severely constrained. Consequently, there is increasing interest in energy-efficient edge AI architectures that support on-chip learning while minimizing memory accesses.

Existing methods to reduce this memory bottleneck often rely on off-chip processing. Offloading computation to the cloud reduces the local RAM requirements for edge devices, saving local power [5]. However, off-chip processing introduces severe issues regarding transmission latency, bandwidth limits, and user privacy [6]. Consequently, to achieve autonomous, real-time edge AI, on-chip processing combined with high-density local memory is the only viable architectural option [7, 8]. To make on-chip processing energy-efficient, Spiking Neural Networks (SNNs) are used. SNNs process information using sparse, discrete events rather than continuous floating-point values, allowing them to replace power-hungry multiplications with simple additions [9].

1.2 Motivation

Training is significantly more energy-intensive than inference because it requires frequent read and write operations to memory for continuous weight updates [3, 10]. Reducing the energy cost of on-chip learning is therefore a critical challenge, particularly for resource-constrained edge and IoT devices. Spiking Neural Networks (SNNs) address this challenge through Spike-Timing-Dependent Plasticity (STDP), a localized and unsupervised learning rule that updates synaptic weights based solely on the relative timing of pre- and post-synaptic spikes [11]. Unlike gradient-based training, STDP performs learning using only local information, eliminating the need to compute, store, and communicate large error gradients across the chip, thereby substantially reducing memory traffic and energy consumption [12].

To further reduce energy consumption, on-chip memory can be operated under ag-

gressive voltage scaling or near-threshold operation [13]. Owing to their sparse, distributed neural representations, Spiking Neural Networks (SNNs) exhibit inherent robustness to hardware imperfections, making them attractive candidates for low-voltage operation [14]. However, as the supply voltage approaches the sub-threshold region, process variations significantly increase the likelihood of permanent memory faults, primarily Stuck-at-0 (SA0) and Stuck-at-1 (SA1) faults [15, 16]. Although these faults can often be mitigated during inference through error correction or by reprogramming affected memory locations [17, 18], such approaches are impractical for on-chip learning, where synaptic weights are continuously updated throughout the training process. Consequently, permanent memory faults directly interfere with the learning dynamics and can severely degrade training performance.

These hardware faults fundamentally disrupt the SNN by locking synaptic weights to minimum or maximum conductance. SA0 faults deaden the connection, which prevents learning on a single synapse but has minimal impact on the broader network due to SNN redundancy [19]. Conversely, SA1 faults lock the synapse to maximum conductance, forcing the receiving neuron to fire continuously at the hardware limit [20]. This hyperactive firing triggers massive lateral inhibition, which permanently suppresses all healthy neurons in the network, paralyzing the STDP process and causing catastrophic accuracy loss during training.

1.3 Contributions

To prevent catastrophic collapse of STDP learning, we propose STDP-DIP (Spike-Timing-Dependent Plasticity with Diagnostic Isolation Pruning), a fault-aware on-chip learning framework for SNNs operating with unreliable near-threshold memory. At system boot, STDP-DIP analyzes the initial memory states to automatically identify neurons affected by SA1 faults. It then selectively prunes the lateral inhibitory outputs of the faulty neurons, effectively isolating the hyperactive activity while preserving normal neuronal dynamics and STDP learning. The main contributions of this thesis are:

1. A formal characterization of SA1 neuron behavior as a function of eDRAM undervolting, mapped via Monte Carlo circuit simulations.

2. The STDP-DIP framework, which structurally isolates hyperactive SA1 neurons to prevent network-wide STDP training collapse.

3. Empirical validation showing that scaled SNNs can seamlessly absorb massive hardware defects, maintaining high accuracy under extreme undervolting without relying on high-overhead combinatorial logic.

The organization of the thesis is as follows: Chapter 2 characterizes the eDRAM weak-cell fault models, STDP dynamics, and visually demonstrates the hardware-induced training collapse. Chapter 3 reviews related work in on-chip learning, DRAM reliability, and fault tolerance. Chapter 4 details the proposed STDP-DIP framework. Finally, Chapter 5 presents the experimental results, followed by the discussion and conclusion in Chapter 6.

Chapter 2

Background

2.1 Spiking Neural Networks and the LIF Neuron Model

Spiking Neural Networks (SNNs) represent the third generation of artificial neural networks, fundamentally differing from traditional deep learning by incorporating time as a primary computational dimension [21]. Rather than passing continuous, high-precision floating-point numbers between layers, SNNs communicate using sparse, binary events known as ”spikes.”

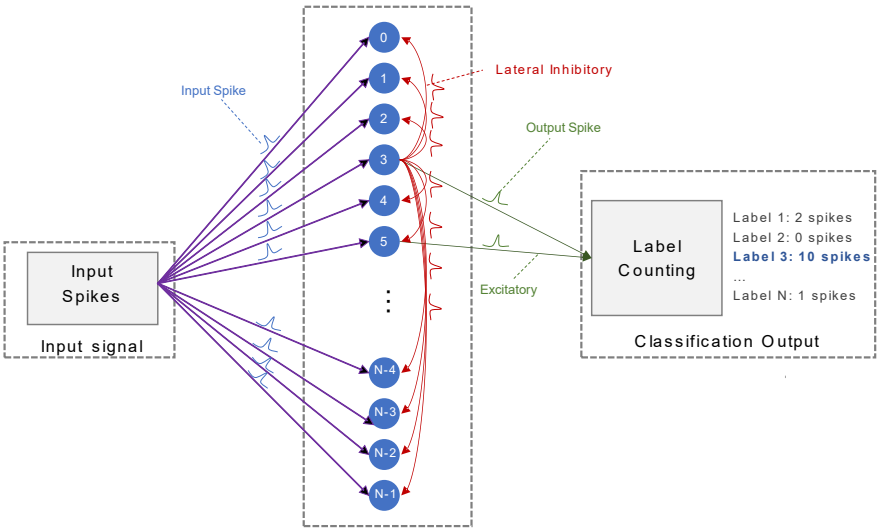


Figure 2.1: A Spiking Neural Network (SNN) topology employing Spike-Timing-Dependent Plasticity (STDP). Input data is encoded into discrete spike trains and transmitted across weighted synapses to an excitatory layer of Leaky Integrate-and-Fire (LIF) neurons, which learn and adapt their feature representations based on localized spike timing.

As illustrated in the macroscopic topology in Figure 2.1, information enters the system via input spikes. These spikes travel along input wires (axons), are multiplied by synaptic weights, and are accumulated by the computational units of the network: Leaky Integrate-and-Fire (LIF) neurons [22].

Consistent with the Diehl and Cook (2015) SNN architecture [11], our network utilizes conductance-based synapses. A neuron acts as a biological capacitor, integrating

incoming spikes into its internal membrane potential (V), which simultaneously "leaks" back to a resting state over time. The membrane potential is governed by:

$$\tau \frac{dV}{dt} = (E_{rest} - V) + g_e(E_{exc} - V) + g_i(E_{inh} - V) \tag{2.1}$$

where τ is the membrane time constant, E_{rest} is the resting potential, E_{exc} and E_{inh} are the equilibrium potentials of excitatory and inhibitory synapses, and g_e and g_i represent the dynamic conductances of those synapses. When V surpasses a dynamic threshold (θ), the neuron emits a discrete binary spike, and its internal potential instantly resets. Synaptic conductances increase instantaneously upon receiving a spike, and decay exponentially otherwise:

$$\tau_{g_e} \frac{dg_e}{dt} = -g_e \tag{2.2}$$

2.2 Spike-Timing-Dependent Plasticity (STDP) and WTA Dynamics

SNNs learn via localized, unsupervised rules. The most prominent of these is Spike-Timing-Dependent Plasticity (STDP) [23, 24]. STDP modifies the synaptic weight (Δw) based strictly on the temporal correlation ($\Delta t = t_{post} - t_{pre}$) of pre- and post-synaptic spikes [11]. When a postsynaptic spike occurs, the synaptic weight w is updated based on an exponential time dependence [11]:

$$\Delta w = \eta(x_{pre} - x_{tar})(w_{max} - w)^{\mu} \tag{2.3}$$

where η is the learning rate, x_{tar} is the target trace value, w_{max} is the maximum allowable weight, and μ controls the weight dependence.

As visualized in Figure 2.2, causality drives STDP. A pre-synaptic spike followed by a post-synaptic spike yields Long-Term Potentiation (LTP), physically strengthening the connection. An acausal firing order triggers Long-Term Depression (LTD). Coupled with STDP is Winner-Take-All (WTA) lateral inhibition. When an excitatory neuron fires, it broadcasts a massive negative voltage penalty to all other excitatory neurons to enforce competition and feature diversification [11].

2.3 Embedded DRAM (eDRAM) and CMOS Leakage Mechanics

To support large weight matrices natively on the processor, designers utilize embedded DRAM (eDRAM) consisting of a 1T1C structure (one access transistor and one storage capacitor) [25]. Because data is stored as a volatile charge, eDRAM must be periodically refreshed. To push processors into ultra-low-power regimes, designers employ Dynamic Voltage and Frequency Scaling (DVFS) and extend the refresh interval [26].

To rigorously characterize the physical manifestation of these failures, we designed and simulated a 1T1C eDRAM bitcell utilizing 45nm Predictive Technology Model (PTM) CMOS parameters [27]. A Monte Carlo (MC) simulation of 1,000 trials was

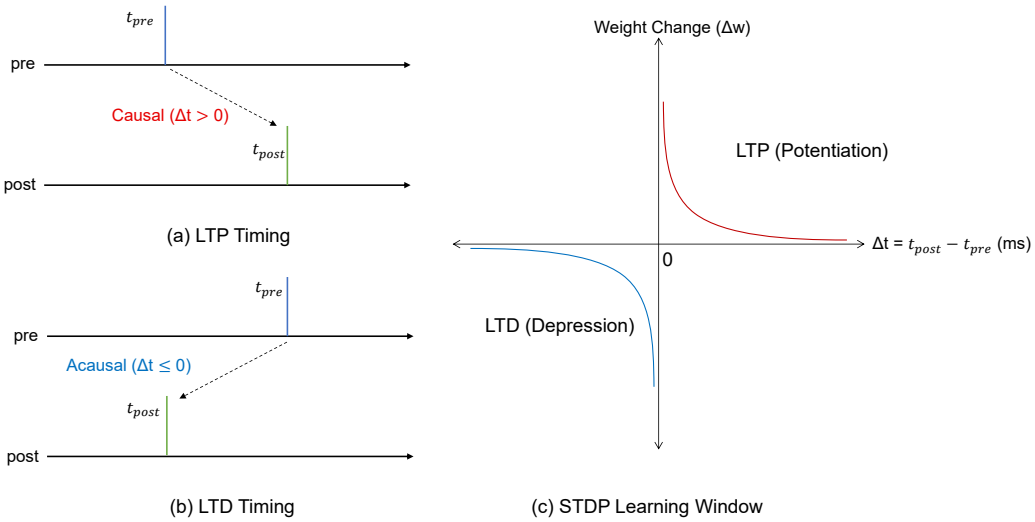


Figure 2.2: Mechanism of STDP weight updates. **(a)** Long-Term Potentiation (LTP) occurs when a pre-synaptic spike causally precedes a post-synaptic spike, strengthening the synapse. Long-Term Depression (LTD) occurs when the post-synaptic spike fires first, weakening the acausal connection. **(b)** The standard STDP learning window mapping the exponential decay of weight updates (Δw) relative to the spike timing difference (Δt).

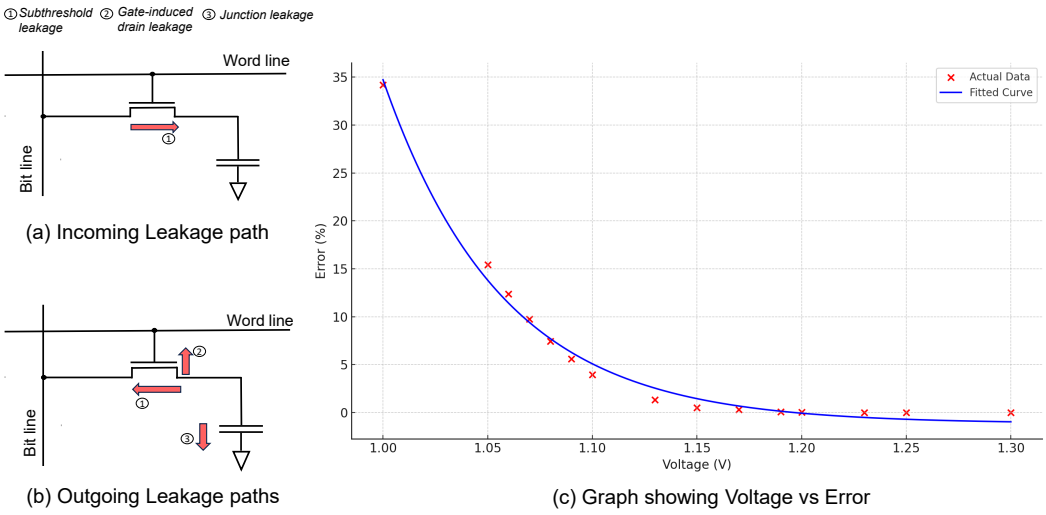


Figure 2.3: **(a)** Circuit-level schematic of a 1T1C eDRAM bitcell illustrating charge leakage paths under aggressive undervolting. **(b)** Empirical Bit Error Rate (BER) extracted from 1,000-trial HSPICE Monte Carlo simulations. As supply voltage is scaled down from $1.30V$ to $1.00V$, sub-threshold leakage accelerates, driving an exponential surge in memory retention failures.

executed in HSPICE to capture the effects of process variations across a voltage sweep from $1.30V$ down to $1.00V$.

As depicted in Figure 2.3, the system is highly stable near nominal voltages ($> 1.20V$). However, aggressive undervolting exposes the weak cells within the memory array, leading to an exponential surge in errors, reaching nearly 35% at $1.00V$. The probability p_b of a bit failing accurately fits an exponential relation to the voltage drop [15]:

$$p_b(V_{dd}) = \frac{\beta_0}{1 + e^{\alpha(V_{dd} - V_{\text{nom}})}} \tag{2.4}$$

2.4 SA1 Faults and the "Dead Neuron" Training Collapse

When weak memory cells fail to retain their charge, they manifest as permanent memory faults. In weight-stationary neuromorphic crossbars, these retention failures manifest as Stuck-at-0 (SA0) or Stuck-at-1 (SA1) faults, locking the cell to the ground or supply voltage (V_{dd}), respectively. As shown in Figure 2.4, SA0 faults deaden the connection, acting as a broken wire. Because SNNs utilize highly distributed neural representations, the network easily absorbs this missing signal [19]. Specifically, a synaptic bit fault leads to continuous neuronal firing only when an SA1 fault afflicts the Most Significant Bit (MSB) of an incoming weight. This MSB SA1 fault locks the synapse to maximum conductance ($W = 1.0$), flooding the post-synaptic neuron with maximal current at every input spike. This causes the membrane potential to instantly saturate and fire relentlessly, heavily corrupting network dynamics [20].

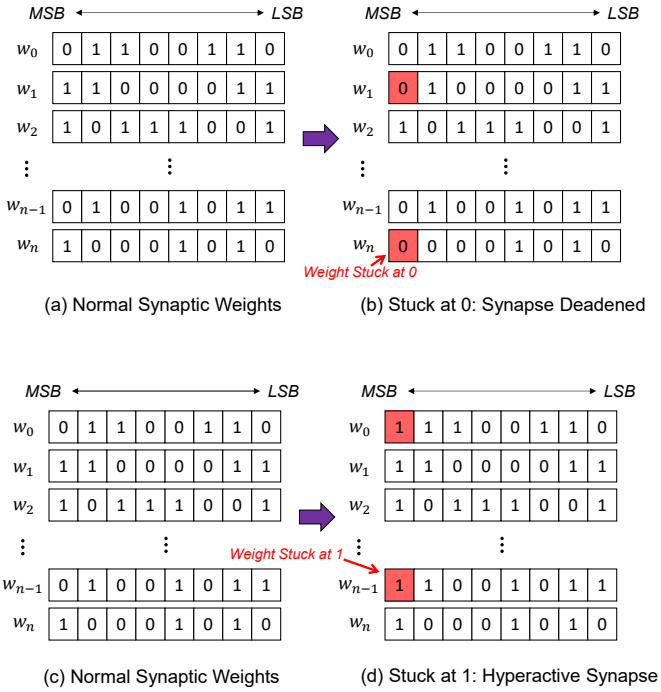


Figure 2.4: Visual representation of binary data corruption in an 8-bit synaptic weight. SA0 faults deaden the connection, while SA1 faults lock the weight bit to a continuous high state, driving hyperactive neuronal behavior.

In this work, we primarily focus on SA1 faults because they are actively destructive. A single hyperactive neuron acts as a Denial-of-Service attack on the entire network via lateral inhibition. Conversely, SA0 faults simply deaden the connection. In real-world environments featuring mixed fault patterns (simultaneous SA0 and SA1 errors), the actively destructive nature of SA1 faults mathematically dominates the network error. An SA0 fault merely prevents learning on a single synapse, whereas a single unmitigated SA1 fault paralyzes the entire WTA layer, making SA1 isolation the critical bottleneck for surviving undervolting.

Traditional software simulators mask these defects because algorithmic variables have no physical limits; if a simulated neuron fires excessively, the software adaptively raises its firing threshold ($\theta \to \infty$) until the neuron is artificially silenced. However, physical neuromorphic circuits are bound by hardware voltage saturation limits. By clamping the adaptive threshold of SA1-afflicted neurons to hardware-realistic boundaries ($\theta = 0.0$), we observe catastrophic training collapse.

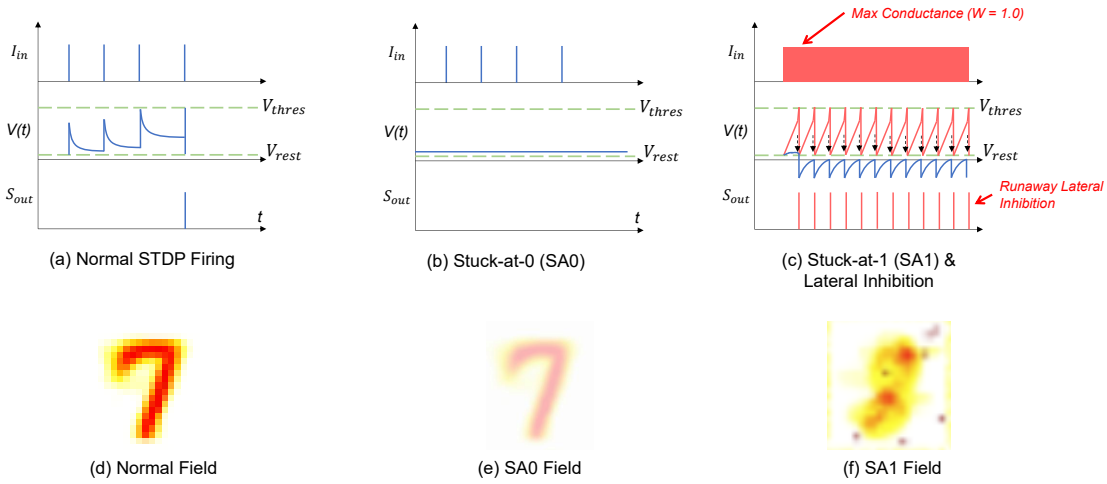


Figure 2.5: Comparative analysis of membrane potential dynamics and receptive field evolution. (a) Normal Operation: Neurons integrate spikes, fire sparsely, and successfully learn feature representations, resulting in a clear receptive field (d). (b) SA0 Fault: The deadened input prevents the membrane potential from reaching the threshold, resulting in faint, unlearned representations (e). (c) SA1 Fault: The faulty neuron fires continuously at the hardware limit, triggering massive WTA lateral inhibition. Neighboring healthy neurons (blue trace) are permanently trapped below V_{rest}, degenerating into ”Dead Neurons” with noisy, unlearned receptive fields (f).

As seen in Figure 2.5(c), because the faulty neuron's input synapses are physically locked to maximum conductance, it fires continuously at the maximum hardware frequency. This hyperactive firing triggers relentless, non-stop lateral inhibition across the entire network. The healthy neurons are subjected to a continuous barrage of inhibitory voltage penalties, permanently suppressing their membrane potentials.

Because these healthy neurons are never permitted to spike, STDP weight updates are never triggered. The healthy population becomes a matrix of ”dead neurons,” their receptive fields remaining in a state of random initialization noise. Consequently, training diverges entirely, and network accuracy collapses to random chance.

Chapter 3

Related Work

3.1 On-Chip Continual Learning in Neuromorphic Hardware

The transition of AI from cloud-centric processing to edge-based autonomous agents has heavily motivated the development of on-chip continual learning frameworks. While commercial and academic neuromorphic chips such as Intel's Loihi [28] and IBM's TrueNorth [29] have demonstrated extreme energy efficiency during inference, the ability to adapt to new environmental stimuli in-situ remains a critical challenge.

Traditional neural network training relies on the backpropagation algorithm, which requires high-precision gradients to be stored and transmitted globally across the network. This is fundamentally incompatible with the memory and power constraints of edge devices [30]. Consequently, hardware implementations heavily favor localized bio-plausible learning rules like Spike-Timing-Dependent Plasticity (STDP) [11]. Recent advancements, such as the *EnsembleSTDP* framework by Hanyu and Dang [12], have introduced decentralized, serverless architectures that allow multiple distributed neuromorphic nodes to train local models independently before merging them. However, because weight updates in STDP occur continuously and dynamically directly on the memory arrays, the training phase is highly sensitive to hardware noise [20]. Ensuring the robustness of these on-chip learning algorithms against underlying hardware instability is an active area of research.

3.2 Memory Voltage Scaling and Reliability Trade-offs

To support the massive synapse arrays required for STDP natively on the processor, modern architectures utilize embedded DRAM (eDRAM) and 3D stacked memory modules [31]. Because static leakage power constitutes a massive proportion of the total energy budget in these dense arrays, designers employ Dynamic Voltage and Frequency Scaling (DVFS) and aggressive refresh reduction policies [13].

However, numerous studies have highlighted the severe reliability trade-offs inherent in this approach. Kang et al. [15] demonstrated that eDRAM arrays suffer from exponential increases in retention failures as supply voltage scales down toward the sub-threshold regime. Similarly, Liu et al. [32] characterized the data retention behavior of modern DRAM devices, showing that manufacturing process variations produce a distinct sub-population of "weak cells." These cells possess abnormally short retention

times that cannot survive extended low-power refresh intervals, introducing permanent Stuck-at-0 and Stuck-at-1 faults into the crossbar [33]. Accepting these errors—rather than attempting to build flawless, power-hungry memory controllers—is widely considered the only viable path toward ultra-low-power neuromorphic design.

3.3 Traditional Memory Fault Tolerance

In traditional Von Neumann architectures, memory faults are heavily mitigated using Error Correction Codes (ECC). Architectures employ SECDED (Single Error Correction, Double Error Detection) or advanced BCH codes by appending redundant parity bits to every memory word [34, 35]. For example, the *ArchShield* framework proposed by Nair et al. [36] utilizes a scalable ECC architecture to tolerate high error rates in DRAM scaling.

While highly effective for general-purpose CPUs, ECC imposes a catastrophic overhead on Spiking Neural Networks. In an SNN, thousands of synaptic weights are accessed and updated every millisecond during the STDP learning phase. Hassan et al. [17] investigated the implementation of ECC directly on neuromorphic architectures and found that the active energy overhead required to encode and decode parity bits for every single synaptic access easily eclipses the static energy saved by undervolting the array.

Alternatively, system-level checkpointing frameworks periodically save the state of the network to non-volatile memory, allowing rollback recovery in the event of power degradation or detected faults [37]. This approach is impractical for autonomous edge devices. Rolling back requires massive data movement over the memory bus, consuming significant energy and halting real-time processing capabilities.

3.4 Fault-Tolerant Neural Network Inference

To avoid the hardware overhead of ECC, substantial research has focused on algorithm-hardware co-design to make neural networks natively resilient to memory faults. Kline et al. [38] proposed the *FLOWER* framework, which utilizes low-overhead bit-level fault-maps to selectively protect only the Most Significant Bits (MSBs) of neural weights. Other approaches rely on heavy quantization, post-training weight pruning, or injecting Gaussian noise into the software simulation during the training phase to force the network to learn robust feature representations [39].

However, these techniques are almost exclusively designed to protect the *inference* phase of previously trained Deep Neural Networks (DNNs). They assume the network has already successfully converged on high-precision cloud servers. They do not address the unique spatial and permanent nature of Stuck-at-1 (SA1) hardware faults [40], nor do they address the problem of ”training collapse” when learning occurs directly on the faulty hardware.

3.5 Emerging SNN Hardware Fault Tolerance

In recent years, literature specifically addressing fault tolerance in Spiking Neural Networks has emerged. Spyrou et al. [20] performed an extensive analysis of neuron

fault tolerance in SNNs, empirically demonstrating that while SNNs degrade gracefully under mild noise, SA1 faults drive uncontrolled neuronal firing that severely disrupts network accuracy.

To mitigate this, Putra et al. proposed the *ReSpawn* [18] and *SoftSNN* [41] frameworks. *ReSpawn* improves fault tolerance by intelligently remapping synaptic weights to avoid faulty memory bitlines, while *SoftSNN* employs a low-cost algorithmic retraining phase to adapt the network around identified soft errors. While these methods show promise, they generally treat faults as static obstacles to be bypassed after a primary training phase.

Our proposed STDP-DIP framework diverges from these works by specifically targeting the *continuous on-chip learning phase*. Drawing inspiration from structural plasticity in biological brains [42], the ability to physically prune hyperactive synapses to preserve cognitive function, STDP-DIP operates without halting the STDP process or relying on external retraining. By physically severing the lateral inhibitory routing of SA1 neurons, STDP-DIP structurally absorbs the hardware failure natively during the learning pipeline.

Chapter 4

Proposed Method: Diagnostic Isolation Pruning (STDP-DIP)

4.1 SNN Topology and Isolation Mechanism

As fault-tolerant systems cannot rely on software thresholds to mask hardware shorts, the STDP-DIP framework utilizes a proactive structural intervention strategy, consisting of Boot-Time Diagnosis and Structural Pruning.

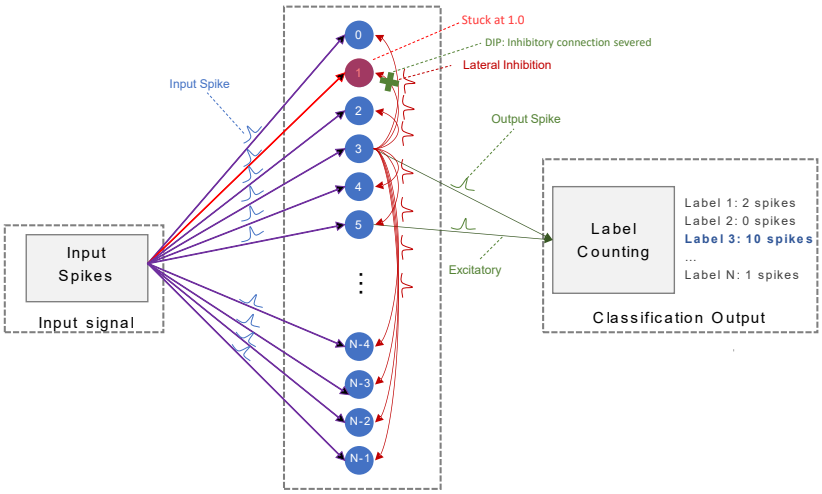


Figure 4.1: Network-level topology of the STDP-DIP framework illustrating the mechanism of training recovery. Input spikes travel through a synapse stuck at maximum conductance ($W = 1.0$), forcing the SA1 excitatory neuron to fire continuously. Without intervention, these hyperactive spikes would flood the inhibitory routing node and broadcast a continuous voltage penalty, paralyzing the healthy neurons. The STDP-DIP mechanism intervenes by structurally severing the faulty neuron's output connection (green 'X'), neutralizing the lateral inhibition threat without disrupting the physical input crossbar.

A Built-In Self-Test (BIST) scans the initial weight matrix before training begins. Because healthy eDRAM weights are initialized to low random values, any bit reading as maximal conductance (1.0) at boot-time is mathematically guaranteed to be a physical hardware short. Traditional software pruning attempts to repair the input side by zeroing out these defective weights. However, this violates the physics of a permanent

hardware short in the eDRAM crossbar array, as the physical wire cannot be algorithmically "un-shorted." Furthermore, simply excluding the faulty neuron from the final classification voting pool is insufficient; during the training phase, the neuron would still fire continuously and trigger network-wide lateral inhibition, halting STDP learning entirely.

Instead, as shown in Figure 4.1, STDP-DIP structurally "prunes" the *inhibitory connection* of the faulty neuron. This unique design choice ensures the faulty neuron continues to fire harmlessly in the background, neutralizing the lateral inhibition threat without violating physical crossbar constraints, thereby allowing healthy neurons to learn normally.

4.2 Algorithmic Implementation of STDP-DIP

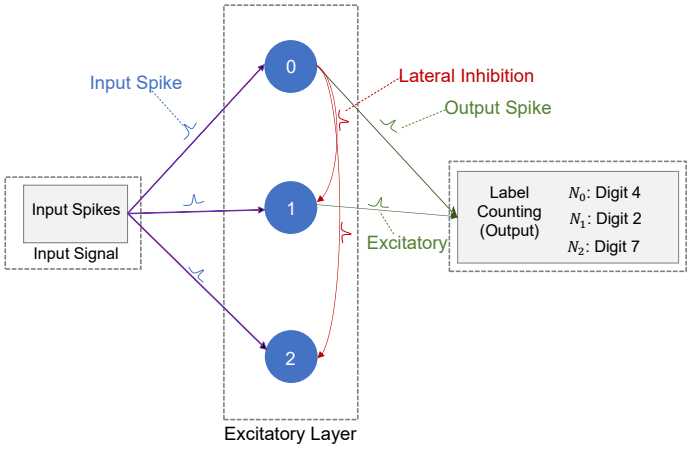
The procedural execution of the STDP-DIP framework bridges the gap between hardware diagnostics and software adaptability. This process is divided into two primary phases, formalized in Algorithm 1. To clarify the interactions between the algorithmic logic and the network's physical state, Figure 4.2 visually tracks this execution across a 3-neuron topology, contrasting normal STDP operation (Figure 4.2a), catastrophic training collapse (Figure 4.2b), and our proposed recovery mechanism (Figure 4.2c).

Phase 1: Boot-Time Hardware Diagnosis: The framework initializes by defining an empty isolation set $\mathcal{P}$ (Algorithm 1, Line 2). This set acts as a persistent hardware defect registry that spans the lifecycle of the network's operation. The concrete detection procedure involves sequentially scanning the memory rows containing the initialized synaptic weights at $t = 0$. Because standard SNN initialization algorithms assign starting weights from a tightly bounded low-conductance distribution, the probability of a healthy weight naturally generating a maximum value (e.g., `11111111`) is statistically near zero. Thus, the possibility of false detection is negligible. Any synapse exhibiting this maximal state at boot-time is deterministically flagged as a physical hardware short.

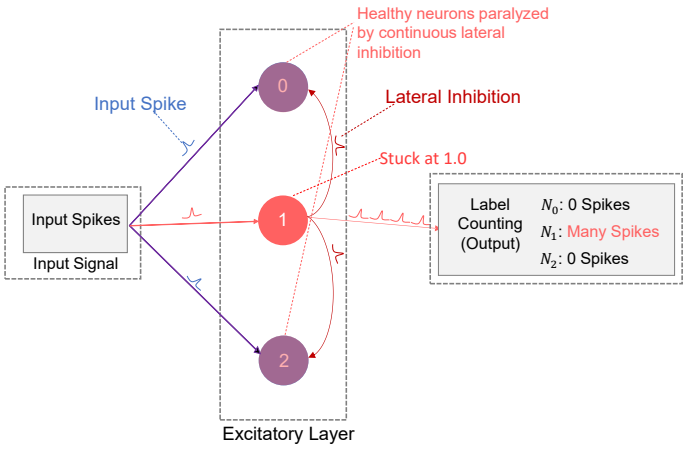
Upon identifying a hyperactive neuron, the algorithm must isolate the threat without violating the physical constraints of the crossbar. While traditional algorithms zero out the defective input weights, this approach is physically impossible in an array with permanent shorts. Therefore, STDP-DIP targets the internal routing architecture. The afflicted neuron's index is appended to $\mathcal{P}$ (Line 6), and its dedicated output connection to the inhibitory routing matrix is structurally severed by setting $\mathbf{W}_{inh}[j, j] = 0.0$ (Line 7). As depicted by the green 'X' marks in Figure 4.2(c), this pruning acts as an open switch, effectively trapping the hyperactive spikes locally inside the neuron and neutralizing the lateral inhibition threat before training commences.

Phase 2: Fault-Aware Learning: During the active STDP training phase (Lines 11–15), the physical constraints of the undervolted memory must be dynamically enforced. The framework continually maps the current supply voltage to a system-wide Bit Error Rate (β) and utilizes the `InjectFaults` module to overwrite the active weight matrix ($\mathbf{W}_{active}$) at every timestep (Lines 12–13). This step guarantees that the simulation obeys hardware reality: the stuck SA1 bits are permanently locked and cannot be "un-learned" or weakened by subsequent STDP updates.

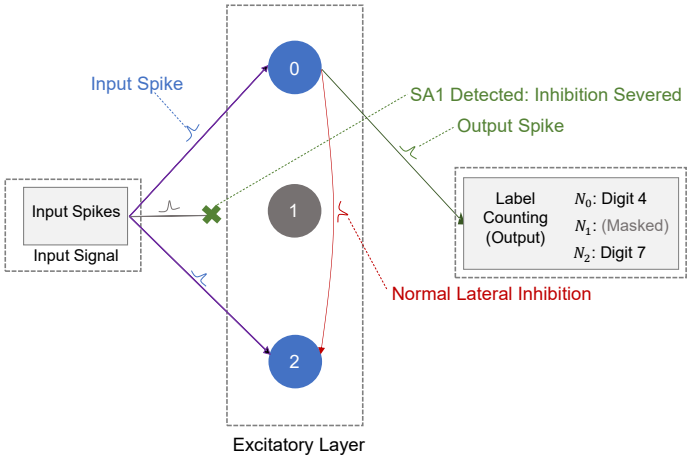
Despite the continuous injection of maximum conductance faults, the network suc-



(a) Normal STDP Operation



(b) Uncorrected SA1 Fault



(c) Corrected using STDP-DIP

Figure 4.2: Architectural execution of the STDP-DIP framework demonstrated on a 3-neuron topology. **(a) Normal:** Healthy LIF neurons receive sparse input spikes, firing and learning via STDP. **(b) Uncorrected:** An SA1 fault locks Neuron 1's input to maximum conductance. It fires continuously, permanently paralyzing healthy Neurons 0 and 2 via continuous lateral inhibition. **(c) STDP-DIP Corrected:** The BIST detects the anomaly and structurally severs Neuron 1's inhibitory connections (green 'X's). Healthy WTA spiking resumes for Neurons 0 and 2, and Neuron 1's outputs are mathematically masked during final classification.

cessfully executes the SNN simulation and the STDP weight updates (Lines 14–15). Because the structural pruning from Phase 1 trapped the faulty neuron's outputs, the SA1 neuron fires harmlessly in the background. Consequently, the healthy neurons (e.g., Neurons 0 and 2 in Figure 4.2c) are liberated from constant suppression. They resume standard Winner-Take-All (WTA) competition, successfully accumulating charge, firing sparse spikes, and undergoing Long-Term Potentiation (LTP) to learn spatial features.

Periodic Labeling and Masking: Because SNNs utilize unsupervised learning, the network does not inherently know which neurons correspond to which digits. Therefore, within Phase 2, the system periodically pauses to recalibrate the neuron-to-class assignments based on proportional spiking activity (Lines 16–17). Left unmitigated, the faulty neurons would inherently dominate this assignment process because they fire at the maximum hardware frequency, generating thousands of useless spikes that would heavily skew the prediction voting pool.

To prevent this predictive corruption, the STDP-DIP algorithm iterates through the previously populated isolation set $\mathcal{P}$ (Line 18) and systematically overwrites the assignments of all faulty neurons with a null label (-1) (Line 19). As visualized in the Label Counting module on the far right of Figure 4.2(c), this digital mask ensures that any spikes emitted by the broken hardware are mathematically discarded during final classification. This final software intervention works in tandem with the structural pruning to fully restore and preserve the network's state-of-the-art accuracy.

Algorithm 1 STDP-DIP: Spike-Timing-Dependent Plasticity with Diagnostic Isolation Pruning

Require: Dataset $\mathcal{D}$, Initial weights $\mathbf{W}_0$, Voltage V_{dd}, Total neurons N
Ensure: Trained, fault-resilient weight matrix $\mathbf{W}^*$
1: *// Phase 1: Boot-Time BIST (Hardware Diagnosis)*
2: $\mathcal{P} \leftarrow \emptyset$ *// Set of pruned/isolated neurons*
3: **for** each neuron $j \in \{1, \ldots, N\}$ **do**
4: $S_j \leftarrow \text{count}(\mathbf{W}_0[:, j] == 1.0)$
5: **if** $S_j > 0$ **then**
6: $\mathcal{P} \leftarrow \mathcal{P} \cup \{j\}$
7: $\mathbf{W}_{inh}[j, j] \leftarrow 0.0$ *// PRUNE: Isolate from inhibitory layer*
8: **end if**
9: **end for**
10: *// Phase 2: Fault-Aware Learning*
11: **for** each sample $(x, y) \in \mathcal{D}$ **do**
12: $\beta \leftarrow \text{MapVoltageToBER}(V_{dd})$
13: $\mathbf{W}_{active} \leftarrow \text{InjectFaults}(\mathbf{W}, \beta)$ *// Insert SA1 shorts*
14: $\text{RunSNN}(x, \mathbf{W}_{active})$
15: $\text{UpdateWeightsSTDP}(\mathbf{W})$ *// Update underlying healthy weights*
16: **if** $\text{step}\%\text{UpdateInterval}(\beta) == 0$ **then**
17: $(\text{Assignments}, \text{Proportions}) \leftarrow \text{AssignLabels}(\text{Spikes}, \text{Labels})$
18: **for** each $j \in \mathcal{P}$ **do**
19: $\text{Assignments}[j] \leftarrow -1$ *// Ignore faulty neurons in classification*
20: **end for**
21: **end if**
22: **end for**
23: **return** $\mathbf{W}$

Chapter 5

Experiments and Results

5.1 Simulation Setup and Multi-Epoch Training

The STDP-DIP framework was evaluated using a Python-based SNN simulation environment powered by BindsNET [43]. SA1 faults were injected statically at the beginning of the simulation and maintained continuously, with the adaptive thresholding mechanism (θ) of the faulty neurons clamped to 0.0.

The network was evaluated on the MNIST dataset using a Poisson Encoder with a simulation time window of 250ms per image. Because unsupervised SNNs require massive data presentations to fully converge, large-scale network sizes (100, 400 and 1000 neurons) were selected. The models were trained continuously across multiple epochs to ensure complete STDP self-organization before evaluation on the 10,000-image test set.

5.2 Large-Scale Robustness and Accuracy Recovery

To validate the diagnostic isolation pruning (DIP) at scale, we conducted a rigorous robustness sweep using the multi-epoch SNN arrays. We expanded the Bit Error Rate (BER) evaluation to cover six magnitudes (0.0001 to 0.2), simulating mild undervolting all the way to extreme, near-threshold operation.

In our fault-injection model, the Bit Error Rate (BER, β) governs the probability of individual memory bit failures across the entire crossbar. However, training collapse is specifically triggered only when these bit-level faults manifest as SA1 errors in the Most Significant Bits (MSBs) of a neuron's incoming synapses. Therefore, the relationship between BER and the number of completely hyperactive neurons is probabilistic. To isolate and rigorously evaluate the network's resilience to WTA collapse, our simulations explicitly fixed the absolute number of fully afflicted neurons (5 and 15) as a direct consequence of the applied BER. This methodology ensures we evaluate the worst-case scenario where bit-flips translate directly into destructive neuronal behavior. Each configuration was executed across three independent random seeds to account for stochastic variance in initialization.

As shown in Table 5.1, the uncorrected baseline models experienced catastrophic training collapse across all parameters. Under standard conditions, accuracy permanently plateaued near random chance ($\sim 10\% - 15\%$). Under extreme fault conditions ($\beta \geq 0.1$), the uncorrected accuracy exhibited higher variance because massive fault in-

Table 5.1: Robustness Results Averages $\pm$ Standard Deviation calculated across 3 independent random seeds.

Neurons	BER (β)	Faulty Neurons	Uncorrected Accuracy (%)	Corrected Accuracy (%)
100	0.001	5	12.18 ± 1.77	77.31 ± 1.03
100	0.01	5	10.05 ± 0.26	77.10 ± 1.35
100	0.1	5	16.97 ± 0.81	77.31 ± 0.44
400	0.001	5	10.94 ± 1.25	89.13 ± 0.28
400	0.01	5	14.14 ± 2.30	89.62 ± 0.22
400	0.1	5	21.47 ± 0.16	89.38 ± 0.81
400	0.001	15	11.94 ± 0.62	88.59 ± 0.07
400	0.01	15	11.15 ± 3.20	87.78 ± 0.09
400	0.1	15	19.84 ± 4.22	88.12 ± 0.46
1000	0.001	5	10.73 ± 0.57	91.77 ± 0.12
1000	0.01	5	11.98 ± 4.44	91.34 ± 0.45
1000	0.1	5	31.91 ± 3.73	92.16 ± 0.85
1000	0.05	5	17.58 ± 1.45	91.12 ± 0.73
1000	0.2	5	14.92 ± 0.88	91.62 ± 0.22
1000	0.001	15	11.88 ± 1.09	91.65 ± 0.36
1000	0.01	15	13.71 ± 5.71	91.31 ± 0.69
1000	0.1	15	20.22 ± 2.01	89.05 ± 0.34
1000	0.05	15	25.99 ± 7.68	91.06 ± 0.71
1000	0.2	15	20.81 ± 2.61	91.86 ± 0.61

jection randomly forces specific digits to dominate the network's chaotic firing, skewing the classification away from pure randomness but failing to achieve generalized learning.

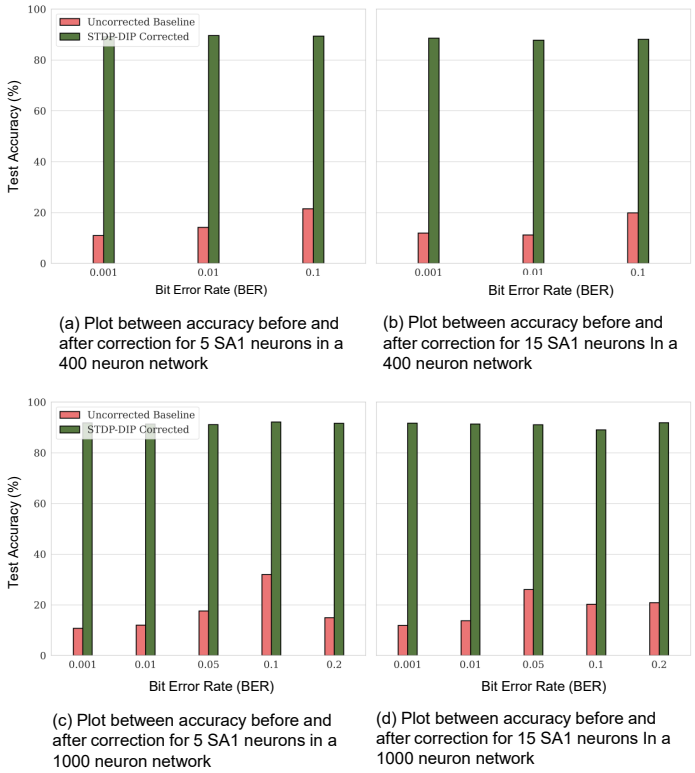
Conversely, the STDP-DIP framework successfully rescued the network in every tested scenario. By detecting and isolating the faulty neurons at boot-time, the corrected networks consistently achieved highly stable accuracies, returning to $\sim 90\%$ for the 400-neuron array and $> 91\%$ for the scaled 1,000-neuron array, as visually summarized in Figure 5.1. Remarkably, the networks maintained this high performance even under extreme near-threshold defect conditions ($\beta = 0.2$), demonstrating that the framework scales robustly to complex neuromorphic arrays.

5.3 Spiking Behavior and Receptive Field Evolution

The efficacy of the STDP-DIP framework is further verified by examining the intermediate spike events.

As established in Figure 5.2, the SA1 fault acts as a "Denial-of-Service" attack on the routing bus. In the uncorrected model, the healthy neurons are entirely suppressed by the hyperactive fault. By pruning the inhibitory output, STDP-DIP restores normal, sparse STDP spiking dynamics across the network.

Because the healthy neurons are allowed to spike, they successfully evolve crisp, highly specialized digit representations over the multiple training epochs (Figure 5.3). The faulty neurons continue to exhibit severe SA1 noise, but because their inhibitory outputs are pruned, they are safely bypassed by the learning algorithm.



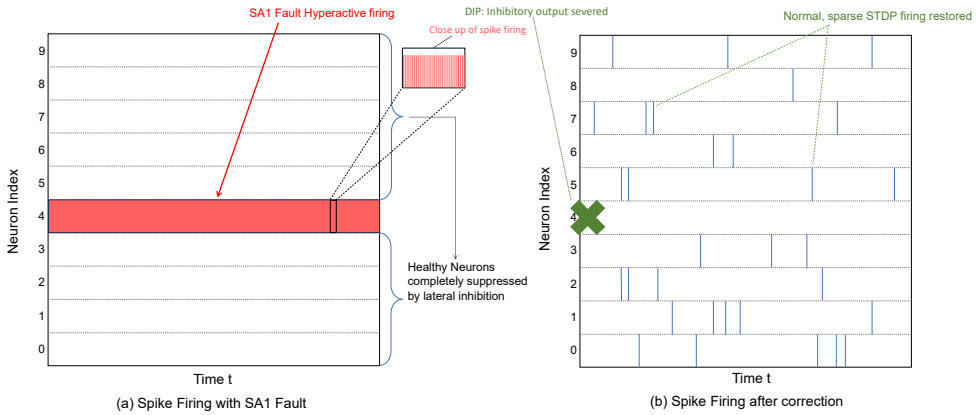
(a) Plot between accuracy before and after correction for 5 SA1 neurons in a 400 neuron network

(b) Plot between accuracy before and after correction for 15 SA1 neurons In a 400 neuron network

(c) Plot between accuracy before and after correction for 5 SA1 neurons in a 1000 neuron network

(d) Plot between accuracy before and after correction for 15 SA1 neurons In a 1000 neuron network

Figure 5.1: Performance recovery of SNNs under varying Bit Error Rates (BER). **(a,b)** The 400-neuron core architecture subjected to 5 and 15 SA1 faults, respectively. **(c,d)** The scaled 1,000-neuron architecture subjected to 5 and 15 SA1 faults. The STDP-DIP framework successfully restores accuracy to nominal bounds across all fault severities and network scales, preventing the catastrophic lateral inhibition collapse observed in the uncorrected baselines.



(a) Spike Firing with SA1 Fault

(b) Spike Firing after correction

Figure 5.2: Spike raster plots of a 10-neuron subset. **(a)** In the uncorrected baseline, Neuron 4 suffers an SA1 fault, firing continuously at the hardware limit and permanently suppressing all healthy neurons. **(b)** STDP-DIP structurally prunes the faulty output (red 'X'), allowing the remaining healthy neurons to resume normal STDP spiking dynamics.

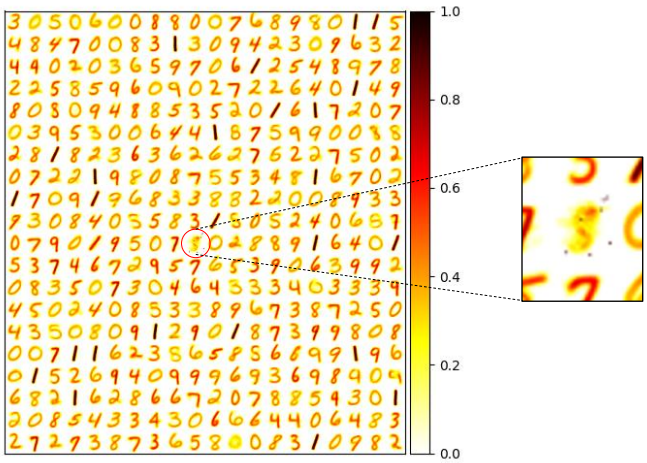


Figure 5.3: Receptive fields of the STDP-DIP corrected network. Healthy STDP evolution occurs normally alongside visually distinct, successfully isolated SA1 faulty neurons (seen as static squares with dark spots).

5.4 Energy Reduction and System-Level Efficiency

A primary advantage of the STDP-DIP framework over existing error-recovery techniques (such as ECC) is its negligible operational overhead. Because BIST evaluation occurs exclusively at boot-time, and the isolation mechanism relies on a single static logic gate per neuron, the framework incurs zero recurring dynamic energy overhead during continuous STDP learning. Consequently, any power saved by undervolting eDRAM arrays translates directly into net energy savings for the processor.

To quantify the energy saved purely within the memory subsystem, we evaluate the extreme undervolting scenario validated in our large-scale robustness sweep. As established previously, the dynamic power consumption of the memory array scales quadratically with the supply voltage ($P \propto V_{DD}^2$). The network demonstrated a stable recovery ($>$91% accuracy) even when the supply voltage was aggressively scaled from a nominal $1.30V$ down to $1.00V$. Applying the quadratic scaling law, the relative dynamic energy consumption at the undervolted state is calculated as:

$$E_{relative} = \left(\frac{V_{low}}{V_{nom}}\right)^2 = \left(\frac{1.00V}{1.30V}\right)^2 \approx 59.17\% \tag{5.1}$$

This represents a direct dynamic energy savings of 40.83% exclusively within the eDRAM memory layers.

To translate these memory-level reductions into overall system-level efficiency, we must account for the memory subsystem's proportion of the total hardware power budget. In modern weight-stationary neuromorphic architectures, synaptic memory access dominates the power profile, accounting for 50% to 75% of the total energy consumed by the system [44]. By applying the 40.83% memory energy reduction to this architectural baseline, the STDP-DIP framework achieves an overall system-wide energy savings of 20.4% to 30.6%. By successfully decoupling hardware reliability from algorithmic performance, STDP-DIP allows neuromorphic processors to operate deep within near-threshold, high-fault regimes, maximizing battery life for edge AI deployments without compromising classification accuracy.

Chapter 6

Discussion and Conclusion

6.1 Discussion

The empirical results clearly demonstrate that algorithmic masking is insufficient for physical hardware faults. Traditional software simulations rely on dynamic homeostasis ($\theta \to \infty$) to silence hyperactive neurons, failing to account for the physical saturation limits of undervolted eDRAM. By clamping the threshold and observing the immediate catastrophic drop to $\sim 10\%$ accuracy, this study proves that lateral inhibition acts as a critical vulnerability vector in neuromorphic hardware. A single unmitigated SA1 fault acts as a "Denial of Service" attack against the entire excitatory layer.

The STDP-DIP framework circumvents this vulnerability by embracing the concept of "acceptable faults." Rather than utilizing power-hungry Error Correction Codes (ECC) to maintain flawless memory states, STDP-DIP leverages the inherent redundancy of large-scale neural arrays. Biological brains routinely experience synaptic death and prune ineffective connections without suffering systemic cognitive failure [42]. Similarly, STDP-DIP isolates the toxic output of defective silicon neurons, allowing the remaining healthy neurons to absorb the computational load.

The hardware overhead of the STDP-DIP approach is exceptionally low compared to traditional fault tolerance mechanisms. Because the Built-In Self-Test (BIST) operates exclusively at boot-time, the energy footprint is limited to a one-time array scan. Furthermore, the isolation mechanism requires only a single static logic gate per neuron to sever the inhibitory output. This passive solution effectively bypasses the continuous, high-energy combinatorial decoding required by ECC during high-frequency STDP memory accesses.

This framework also establishes that unmitigated hardware errors are fundamentally more destructive during the training phase than the inference phase. While inference networks can tolerate corrupted weights through degraded precision, faults during training actively corrupt the structural plasticity of the network, preventing feature learning entirely. Protecting the STDP learning phase is therefore a mandatory prerequisite for deploying fully autonomous edge computing nodes capable of continuous in-situ adaptation.

Future research must explore the generalization of STDP-DIP to more complex network topologies and datasets. Applying this framework to deep convolutional SNNs or multi-cluster Network-on-Chip (NoC) implementations may require granular isolation mechanisms at the inter-router level, as hyperactive firing could propagate across cluster boundaries. Additionally, validating the framework on physical silicon will be

necessary to assess dynamic thermal propagation and complex mixed-fault behaviors under real-world edge environments.

6.2 Conclusion

This thesis presented the Spike-Timing-Dependent Plasticity with Diagnostic Isolation Pruning (STDP-DIP) framework to address the critical "Dead Neuron" training collapse caused by memory undervolting in ultra-low-power SNNs. By characterizing the Bit Error Rates of eDRAM weak cells under aggressive voltage scaling, we demonstrated that Stuck-at-One (SA1) faults cause hyperactive neuronal firing. This hyperactivity permanently paralyzes unsupervised STDP learning via runaway lateral inhibition.

To mitigate this training collapse, STDP-DIP employs a low-overhead Boot-Time Built-In Self-Test (BIST) to blindly detect permanent hardware shorts. In contrast to simple faulty-neuron exclusion which ignores the neuron during inference but fails to prevent pathological lateral inhibition during training, or standard synaptic pruning which incorrectly assumes defective inputs can be algorithmically reprogrammed, STDP-DIP structurally absorbs the hardware failure. Rather than attempting to unshort the physical input matrix via redundant logic, the framework structurally prunes the faulty neuron's output connection to the inhibitory layer. This strategic intervention successfully neutralizes the lateral inhibition threat while adhering to physical hardware constraints. Crucially, the BIST overhead is exceptionally low, requiring only a one-time $O(N)$ memory read penalty at system boot, while the physical isolation relies on a single passive logic gate per neuron. This substantiates our claim that STDP-DIP is vastly more energy-efficient than traditional Error Correction Codes (ECC), which require continuous, power-hungry combinatorial decoding for every single synaptic read and write operation during high-frequency STDP updates.

Experimental results on large-scale SNNs trained over multiple epochs validated the robustness of this approach. Under extreme defect conditions, the uncorrected networks consistently failed at random chance levels ($\sim 10\%$). By implementing STDP-DIP, the networks demonstrated a complete recovery, restoring nominal state-of-the-art accuracy ($>91\%$) across all tested fault severities without disrupting the underlying learning dynamics.

While our experiments successfully validate STDP-DIP on a 1,000-neuron SNN using the MNIST dataset, scaling this approach presents important considerations for future work. Applying this framework to more complex datasets (e.g., CIFAR-10 or ImageNet) and deeper convolutional SNNs may require more granular isolation logic. In highly layered or multi-cluster Network-on-Chip (NoC) implementations, hyperactive firing may propagate across cluster boundaries, requiring isolation mechanisms at the inter-router level rather than solely at the local lateral inhibitory layer. Furthermore, future work should validate the framework on physical silicon implementations to assess dynamic thermal propagation and complex mixed-fault behaviors (simultaneous SA0 and SA1 conditions) under real-world edge environments.

Ultimately, this framework proves that neuromorphic fault tolerance does not require heavy error-correction overhead. By leveraging bio-inspired structural plasticity, we enable reliable, heavily undervolted on-chip learning, advancing sustainable and carbon-neutral edge AI.

References

[1] S. Luccioni, Y. Jernite, and E. Strubell, "Power hungry processing: Watts driving the cost of ai deployment?" in *Proceedings of the 2024 ACM conference on fairness, accountability, and transparency*, 2024, pp. 85–99.

[2] J. McDonald, B. Li, N. Frey, D. Tiwari, V. Gadepally, and S. Samsi, "Great power, great responsibility: Recommendations for reducing energy for training language models," in *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022, pp. 1962–1970.

[3] A. Gholami, Z. Yao, S. Kim, M. W. Mahoney, and K. Keutzer, "Ai and memory wall," *IEEE Micro*, vol. 44, no. 1, pp. 10–18, 2024.

[4] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "Processing data where it makes sense: Enabling in-memory computation," *Microprocessors and Microsystems*, vol. 67, pp. 28–41, 2019.

[5] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[6] K. Muhammad, A. Ullah, J. Lloret, J. Del Ser, and V. H. C. de Albuquerque, "Deep learning for safe autonomous driving: Current challenges and future directions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 7, pp. 4316–4336, 2020.

[7] J. Wang, K. N. Dang, and A. B. Abdallah, "Scaling deep-learning pneumonia detection inference on a reconfigurable self-contained hardware platform," in *2023 6th International Conference on Electronics Technology (ICET)*. IEEE, 2023, pp. 1116–1121.

[8] A. B. Abdallah and K. N. Dang, *Neuromorphic computing principles and organization*. Springer, 2022.

[9] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.

[10] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2014, pp. 10–14.

[11] P. U. Diehl and M. Cook, "Unsupervised learning of digit recognition using spike-timing-dependent plasticity," *Frontiers in Computational Neuroscience*, vol. 9, p. 99, 2015.

[12] Y. Hanyu and K. N. Dang, "Ensemblestdp: Distributed in-situ spike timing dependent plasticity learning in spiking neural networks," in *2024 IEEE 17th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*. IEEE, 2024, pp. 434–440.

[13] P. Koutsovasilis *et al.*, "Dynamic undervolting to improve energy efficiency on multicore x86 cpus," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 12, pp. 2851–2864, 2020.

[14] P. Panda, S. A. Aketi, and K. Roy, "Toward scalable, efficient, and accurate deep spiking neural networks with backward residual connections, stochastic softmax, and hybridization," *Frontiers in Neuroscience*, vol. 14, p. 653, 2020.

[15] U. Kang, D. Lee, and Y. Kim, "Co-architecting edram and sram arrays for energy-efficient reliable on-chip memory," *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*, pp. 372–383, 2014.

[16] E.-I. Vatajelu, G. Di Natale, and L. Anghel, "Special session: Reliability of hardware-implemented spiking neural networks (SNN)," in *2019 IEEE 37th VLSI Test Symposium (VTS)*. IEEE, 2019, pp. 1–8.

[17] M. S. Hassan, P. Dattilo, and A. Akoglu, "A novel implementation methodology for error correction codes on a neuromorphic architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 12, pp. 4706–4720, 2023.

[18] R. V. W. Putra, M. A. Hanif, and M. Shafique, "Respawn: Energy-efficient fault-tolerance for spiking neural networks considering unreliable memories," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–9.

[19] C. D. Schuman, J. P. Mitchell, R. M. Patton, T. E. Potok, and J. S. Plank, "Resilience and robustness of spiking neural networks for neuromorphic systems," *2020 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–10, 2020.

[20] T. Spyrou, S. A. El-Sayed, E. Afacan, L. A. Camuñas-Mesa, B. Linares-Barranco, and H.-G. Stratigopoulos, "Neuron fault tolerance in spiking neural networks," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 743–748.

[21] W. Maass, "Networks of spiking neurons: the third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[22] W. Gerstner and W. M. Kistler, "Spiking neuron models: Single neurons, populations, plasticity," 2002.

[23] G.-q. Bi and M.-m. Poo, "Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type," *Journal of Neuroscience*, vol. 18, no. 24, pp. 10 464–10 472, 1998.

[24] N. Caporale and Y. Dan, "Spike timing–dependent plasticity: a hebbian learning rule," *Annual Review of Neuroscience*, vol. 31, no. 1, pp. 25–46, 2008.

[25] S. Koppula, L. Orosa, A. G. Yağlıcçı, R. Azizi, T. Shahroodi, K. Kanellopoulos, and O. Mutlu, "Eden: Enabling energy-efficient, high-performance deep neural network inference using approximate dram," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019, pp. 166–181.

[26] I. Bhati, Z. Chishti, S.-L. Lu, and B. Jacob, "Flexible auto-refresh: Enabling scalable and energy-efficient dram refresh reductions," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, 2015, pp. 235–246.

[27] W. Zhao and Y. Cao, "New generation of predictive technology model for early design exploration," *IEEE Transactions on Electron Devices*, vol. 53, no. 11, pp. 2816–2823, 2006.

[28] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Osikowicz, and S. Sengupta, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[29] F. Akopyan, J. Sawada, A. Cassidy, R. Alvarez-Icaza, J. Arthur, P. Merolla, N. Imam, Y. Nakamura, P. Datta, and G.-J. Nam, "Truenorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 10, pp. 1537–1557, 2015.

[30] M. Bouvier, A. Valentian, T. Mesquida, F. Rummens, M. Reyboz, E. Vianello, and E. Beigne, "Spiking neural networks hardware implementations and challenges: A survey," *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 15, no. 2, pp. 1–35, 2019.

[31] B. Black, M. Annavaram, N. Brekelbaum, J. Deaver, L. Jiang, G. H. Loh, D. McCullen, A. Nelson, B. Rychlik, and C. Weisell, "Die stacking (3d) microarchitecture," in *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*. IEEE, 2006, pp. 469–479.

[32] J. Liu, B. Jaiyen, Y. Kim, C. Wilkerson, and O. Mutlu, "An experimental study of data retention behavior in modern dram devices: Implications for retention time profiling mechanisms," *ACM SIGARCH Computer Architecture News*, vol. 41, no. 3, pp. 60–71, 2013.

[33] K. Saino, S. Horiba, Y. Uchiyama, Y. Takaishi, M. Takenaka, T. Uchida, Y. Takada, K. Koyama, H. Miyake, and C. Hu, "Impact of gate-induced drain leakage current on the tail distribution of dram data retention time," in *International Electron Devices Meeting 2000. Technical Digest. IEDM*. IEEE, 2000, pp. 837–840.

[34] W. W. Peterson and E. J. Weldon, Jr., *Error-correcting codes*, 2nd ed. MIT Press, 1972.

[35] D. H. Yoon and M. Erez, "Virtualized and flexible ecc for main memory," in *Proceedings of the Fifteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2010, pp. 397–408.

[36] P. J. Nair, D.-H. Kim, and M. K. Qureshi, "ArchShield: Architectural framework for assisting DRAM scaling by tolerating high error rates," *ACM SIGARCH Computer Architecture News*, vol. 41, no. 3, pp. 72–83, 2013.

[37] D. Balsamo, A. S. Weddell, A. Das, A. R. Arreola, D. Brunelli, B. M. Al-Hashimi, G. V. Merrett, and L. Benini, "Hibernus++: A Self-Calibrating and Adaptive System for Transiently-Powered Embedded Devices," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 12, pp. 1968–1980, 2016.

[38] D. Kline, Jr., J. Zhang, R. G. Melhem, and A. K. Jones, "Flower and fame: A low overhead bit-level fault-map and fault-tolerance approach for deeply scaled memories," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 356–368.

[39] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021.

[40] E. I. Muehldorf, "Fault clustering: Modeling and observation on experimental lsi chips," *IEEE Journal of Solid-State Circuits*, vol. 10, no. 4, pp. 237–244, 1975.

[41] R. V. W. Putra, M. A. Hanif, and M. Shafique, "Softsnn: Low-cost fault tolerance for spiking neural network accelerators under soft errors," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 151–156.

[42] G. Chechik, I. Meilijson, and E. Ruppin, "Synaptic pruning in development: a computational account," *Neural Computation*, vol. 10, no. 7, pp. 1759–1777, 1998.

[43] H. Hazan, D. J. Saunders, H. Khan, D. Patel, D. T. Sanghavi, and H. T. Siegelmann, "Bindsnet: A machine learning-oriented spiking neural networks library in python," *Frontiers in Neuroinformatics*, vol. 12, p. 89, 2018.

[44] N.-D. Nguyen, A. B. Ahmed, A. B. Abdallah, and K. N. Dang, "Power-aware neuromorphic architecture with partial voltage scaling 3-d stacking synaptic memory," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 12, pp. 2016–2029, 2023.